# RECONNAISSANCE & ENUMERATION

## TryHackMe Learning & Practical Report

**Prepared by:** Kaustubh Ashish Bangar

**Focus:** Web Application Security / WAPT Foundations

**Task:** Task 2 — Reconnaissance & Enumeration

**Rooms Covered:** Passive Reconnaissance, Active Reconnaissance, Content Discovery

A comprehensive technical report documenting the methodologies, theoretical concepts, and practical exercises completed for the Reconnaissance and Enumeration phase of the IqraSec Academy WAPT Internship Phase 1.

## 1. Introduction

Reconnaissance is the mandatory preliminary survey phase in modern cybersecurity frameworks, such as the Unified Kill Chain. Before a security analyst can identify or exploit vulnerabilities within a web application, they must first systematically map the target's external digital footprint, discover exposed technologies, and identify hidden attack vectors. Reconnaissance is strictly divided into two primary categories: Passive Reconnaissance and Active Reconnaissance.

Passive reconnaissance relies exclusively on gathering intelligence from publicly available, third-party sources (Open-Source Intelligence, or OSINT) without sending any direct traffic to the target's infrastructure, thereby remaining completely stealthy. Active reconnaissance, by contrast, requires direct engagement and packet transmission to the target network, carrying a higher risk of logging and detection by Intrusion Detection Systems (IDS) or Web Application Firewalls (WAF). Content Discovery bridges these domains by actively probing the application layer using manual analysis and automated fuzzing to map out unadvertised directories, hidden APIs, and administrative interfaces.

This consolidated report documents the theoretical principles and practical techniques executed across three dedicated TryHackMe modules: Passive Reconnaissance, Active Reconnaissance, and Content Discovery.

## 2. Objective

The primary objective of this overarching task is to master the techniques required to thoroughly map a web application's external attack surface prior to exploitation.

Specific learning and practical objectives include:

1. **Passive OSINT Gathering:** To extract domain registration details using the legacy WHOIS protocol and its modern secure successor, RDAP.
2. **DNS Enumeration:** To query public resolvers for domain records (A, AAAA, MX, TXT) using command-line utilities like `dig` and `nslookup`.
3. **Subdomain Discovery:** To map organizational infrastructure passively using aggregated DNS datasets (DNSDumpster) and Certificate Transparency (CT) logs (`crt.sh`).

4. **Service Profiling:** To utilize specialized device search engines like Shodan.io to discover exposed ports, hosting providers, and software versions without direct interaction.
5. **Active Network Probing:** To trace network routing pathways and verify host availability using `ping` and `traceroute`.
6. **Banner Grabbing:** To manually extract software version information directly from target service daemons using `telnet` and `netcat` (`nc`).
7. **Content Discovery:** To systematically uncover hidden application functionality, administrative login pages, and unlinked directories using manual techniques and automated tools like Gobuster.

## 3. Concept / Understanding

### *3.1 Passive Reconnaissance Concepts*

Passive reconnaissance minimizes exposure by leveraging data that organizations inevitably leak into the public domain through standard internet operations.

- **WHOIS and RDAP:** WHOIS is a legacy TCP-based protocol (port 43) utilized to query domain registrars for ownership details, registration dates, and authoritative name servers. Due to strict international privacy regulations like GDPR, personal registrant data is now heavily redacted. Consequently, the Registration Data Access Protocol (RDAP) has emerged as the modern successor. RDAP utilizes HTTPS for secure transmission, returns data in structured, machine-readable JSON formats, and provides robust internationalization support. Analysts utilize these tools to identify domain expiration dates (useful for social engineering) or historical infrastructure shifts.
- **DNS Resolution:** The Domain Name System (DNS) is responsible for translating human-readable domain names into routable IP addresses. By querying public resolvers (such as Cloudflare's `1.1.1.1` or Google's `8.8.8.8`), a penetration tester can extract DNS records without alerting the target organization. Key targets include `A` records (IPv4 addresses), `MX` records (Mail Exchange servers), and `TXT` records (arbitrary text, frequently used to hold SPF, DMARC, or third-party domain verification strings). The modern `dig` utility is preferred over the older `nslookup` for these queries due to its comprehensive output and scripting capabilities.

- **Subdomain Enumeration:** Organizations frequently deploy vulnerable, outdated, or staging environments on unadvertised subdomains (e.g., `dev.example.com`). These are mapped passively using tools like DNSDumpster, which aggregates public DNS data from search engine caches. The most authoritative passive method relies on Certificate Transparency (CT) logs, accessible via `crt.sh`. Because CT logs mandate the public recording of every issued SSL/TLS certificate, querying the Subject Alternative Name (SAN) fields easily exposes hidden organizational subdomains without a single packet hitting the target's servers.
- **Shodan.io Device Indexing:** While standard search engines index web page content, Shodan.io actively scans the internet to index connected devices. It categorizes open ports, grabs service banners, and tags exposed infrastructure. Security analysts use Shodan to determine a target's hosting provider (e.g., AWS, Cloudflare), geographic location, and specific server software (like Apache or Nginx) completely passively, filtering queries by organization name or IP address.

### *3.2 Active Reconnaissance Concepts*

Active reconnaissance shifts the methodology toward direct interaction. This phase is analogous to physically testing the locks on a target building and inherently carries the risk of triggering security alerts.

- **Ping (ICMP):** The `ping` utility utilizes the Internet Control Message Protocol (ICMP) Echo Request to verify if a target host is actively responding on the network. While foundational, many modern enterprise firewalls explicitly drop incoming ICMP traffic to minimize their external footprint.
- **Traceroute:** The `traceroute` utility maps the exact network topology and router hops a packet traverses to reach its destination. It functions by manipulating the Time-to-Live (TTL) field of IP packets. Each router that drops an expired packet returns an ICMP Time Exceeded message, revealing its IP address. This is critical for identifying the presence of intermediary devices like Web Application Firewalls (WAFs) or load balancers.
- **Banner Grabbing (Telnet & Netcat):** Connecting directly to an open port using basic TCP clients like `telnet` or `netcat` allows analysts to interact directly with the raw service daemon. When a connection is established, the service frequently responds with

a "banner" containing its identity (e.g., `Server: Apache/2.4.61`). Extracting this exact version number is highly critical, as it can be immediately cross-referenced against vulnerability databases (like CVE or Exploit-DB) to identify potential remote code execution flaws.

### *3.3 Content Discovery Concepts*

Web applications frequently conceal sensitive functionality, such as API endpoints, backup files, configuration environments, or administrative panels, assuming that if a link is not published, it cannot be found (Security through Obscurity). Content discovery systematically defeats this assumption using three primary approaches:

- **Manual Discovery:** Analysts manually inspect predictable locations. The `robots.txt` file dictates rules for automated web crawlers and frequently contains explicit `Disallow:` directives pointing directly to sensitive administrative directories that the owner wishes to hide from search engines. Similarly, the `sitemap.xml` file provides an architectural map of the site intended for SEO purposes, often leaking unlinked application areas. Furthermore, inspecting raw HTTP Response Headers (using tools like `curl`) can reveal custom framework flags (e.g., `X-Powered-By`) that identify the underlying backend technology.
- **OSINT Content Discovery:** Leveraging external sources to find content. This includes Google Dorking (using advanced search operators to find indexed login portals or leaked documents), querying the Wayback Machine for historical, unpatched versions of the site, and searching GitHub repositories for hardcoded credentials or API keys mistakenly committed to public version control.
- **Automated Discovery (Fuzzing):** Utilizing high-speed automated tools like Gobuster, Dirb, or Ffuf paired with extensive wordlists (such as SecLists). These tools rapidly request thousands of potential URIs per second. If the target server responds with a `200 OK` (Found) or `403 Forbidden` (Access Denied) status code, the tool successfully identifies the existence of a hidden directory, file, subdomain, or virtual host.

## 4. Practical Work

The practical application phase involved utilizing both passive and active methodologies against simulated TryHackMe targets to map infrastructure, extract configuration data, and discover hidden web content.

**Passive Reconnaissance Execution:**

- Executed a DNS query against the TXT records of `thmlabs.com` using the `nslookup/dig` utilities, successfully uncovering a hidden verification flag: `THM{a5b83929888ed36acb0272971e438d78}`.
- Utilized the Shodan.io search engine to filter and analyze the global footprint of specific web servers. I identified the United States as the top host globally for accessible Apache servers. Additionally, I documented common alternative port configurations, discovering port 8080 as the third most common for Apache, and port 80 as the primary default for Nginx infrastructure.

**Active Reconnaissance Execution:**

- Analyzed dynamic `traceroute` output against target infrastructure to document external network routing. By examining Traceroute A and Traceroute B, I successfully identified the specific IP addresses of the final router hops (`100.92.9.83` and `99.83.89.19`) immediately preceding the target domain, mapping a 25-hop network chain between the source and the target system.
- Performed active banner grabbing by initiating a raw TCP connection to an operational target VM on port 80 utilizing the `telnet` client. By interacting with the daemon, I successfully intercepted the server response header, confirming the running software as Apache and precisely identifying its version as `2.4.61`.

**Content Discovery Execution:**

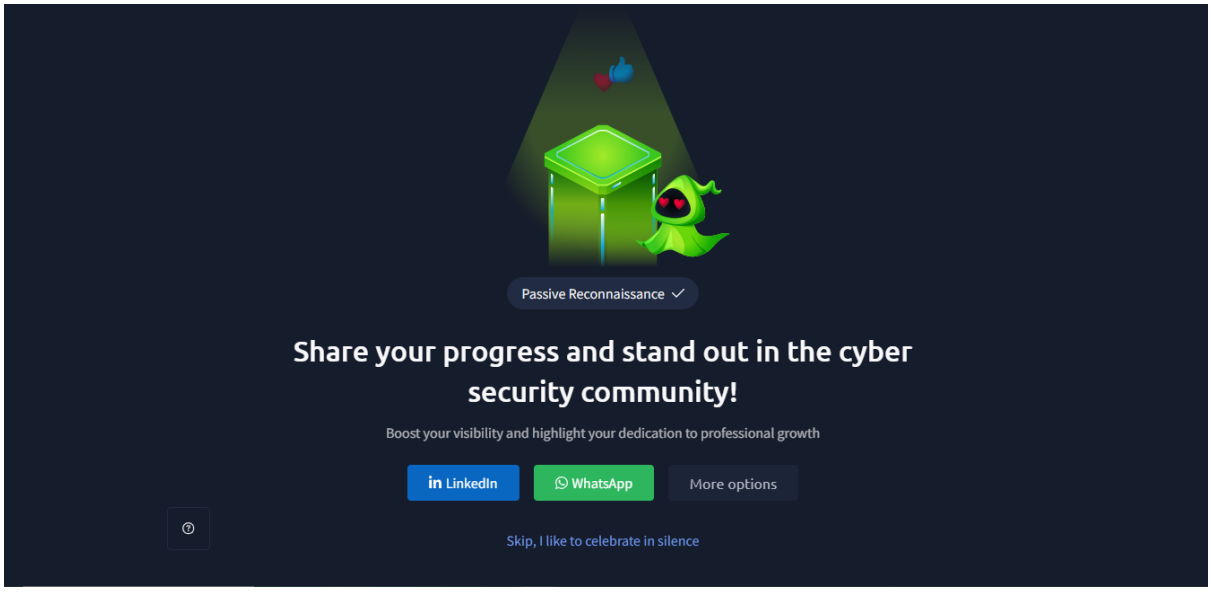
- Conducted manual file discovery by inspecting the target's `robots.txt` file, successfully identifying the `/staff-portal` directory which was explicitly disallowed from web crawler indexing.
- Inspected the target's `sitemap.xml` file, which revealed the existence of a hidden application path named `/s3cr3t-area`.
- Utilized the `curl http://$ip -v` command-line utility to manually perform an HTTP verb request and inspect the raw response headers returned by the server. This analysis successfully revealed a custom security header flag (`X-FLAG: THM{HEADER_FLAG}`) alongside the `X-Powered-By: THM-Framework` header, running on an `nginx/1.18.0 (Ubuntu)` backend.

## 5. Tools Used

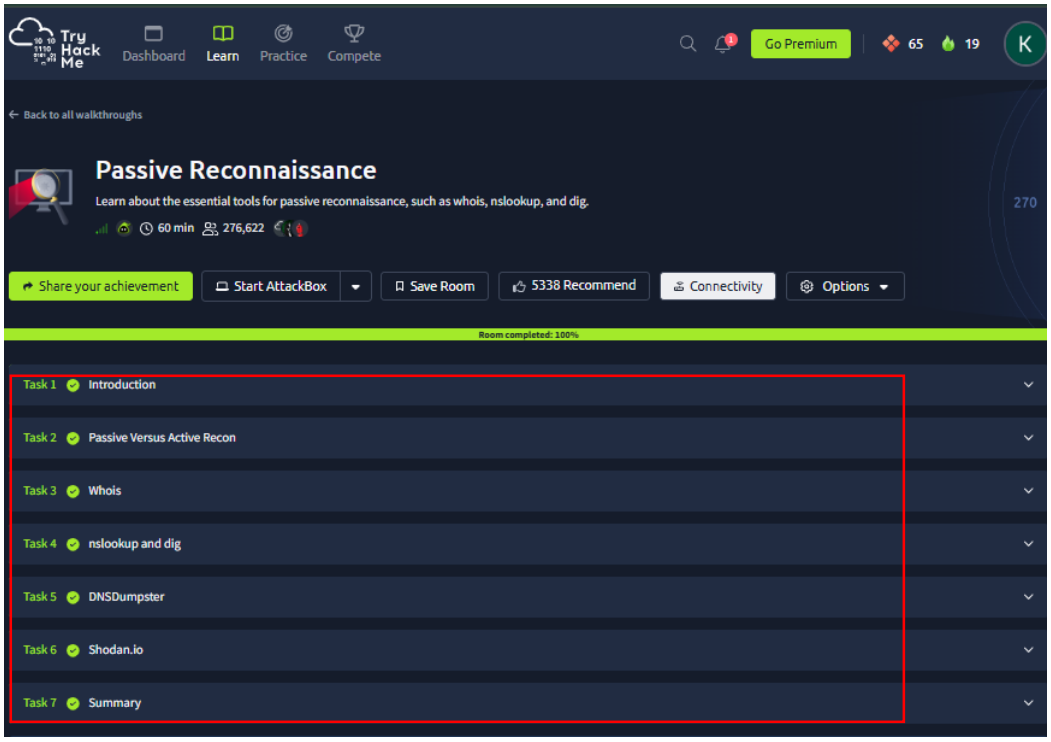
- **Passive OSINT / Web Tools:** Shodan.io, DNSDumpster, crt.sh (Certificate Transparency Logs), Wappalyzer, Wayback Machine, GitHub.
- **Network & DNS Command Line Utilities:** `whois`, `dig`, `nslookup`, `ping`, `traceroute`, `telnet`, `nc` (Netcat), `curl`.
- **Automated Fuzzing:** Gobuster (Subdomain, Directory, and Virtual Host enumeration).
- **Environment:** TryHackMe Browser-based AttackBox.

## 6. Evidence and Screenshots

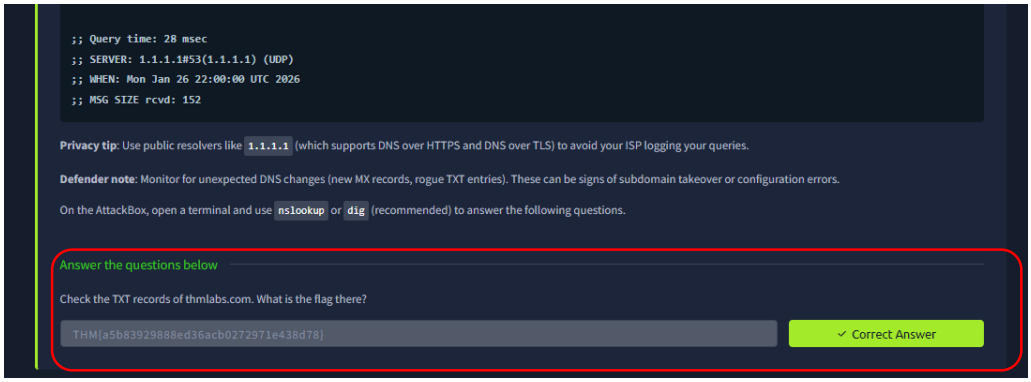
*All relevant visual evidence documenting the successful completion of these exercises is stored in the designated* `Kaustubh_Bangar_WAPT_Phase-1/2.Recon-Enumeration/` *submission directory*



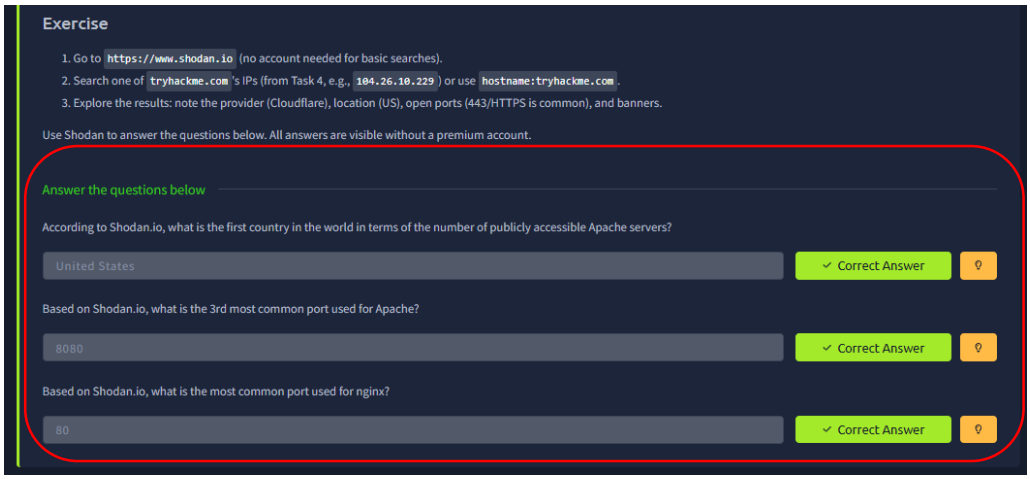

- **Fig. 2.1:** TryHackMe Passive Reconnaissance room completion certificate.



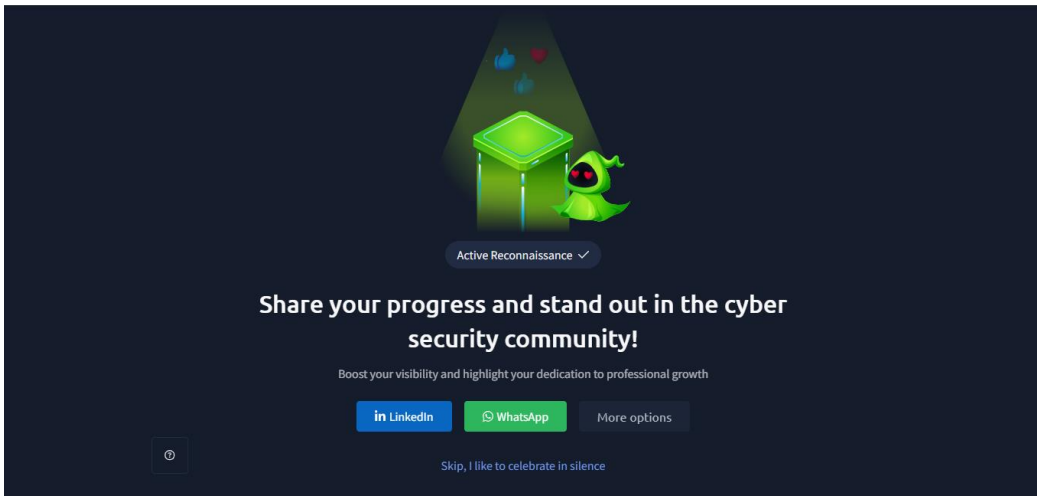

- **Fig. 2.2:** Passive Reconnaissance task dashboard confirming full module completion.



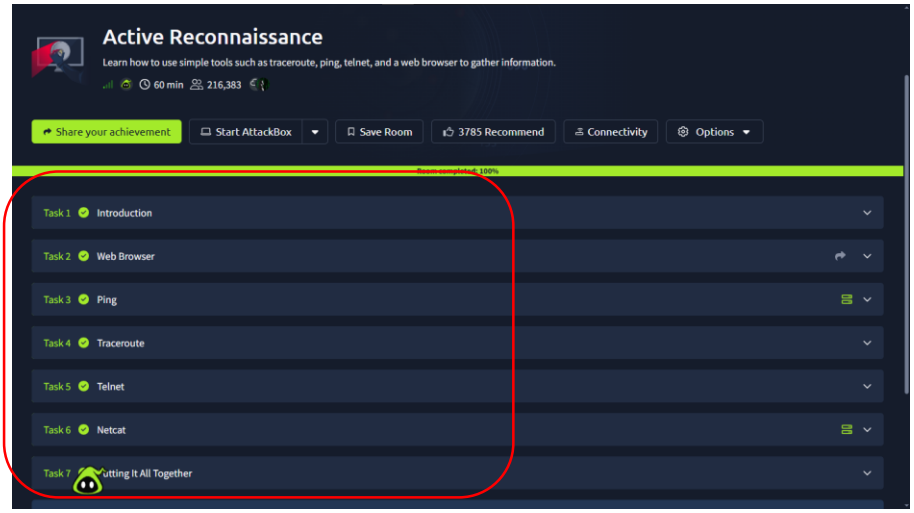

- **Fig. 2.3:** Successful execution of DNS `TXT` record queries revealing the THM verification flag.



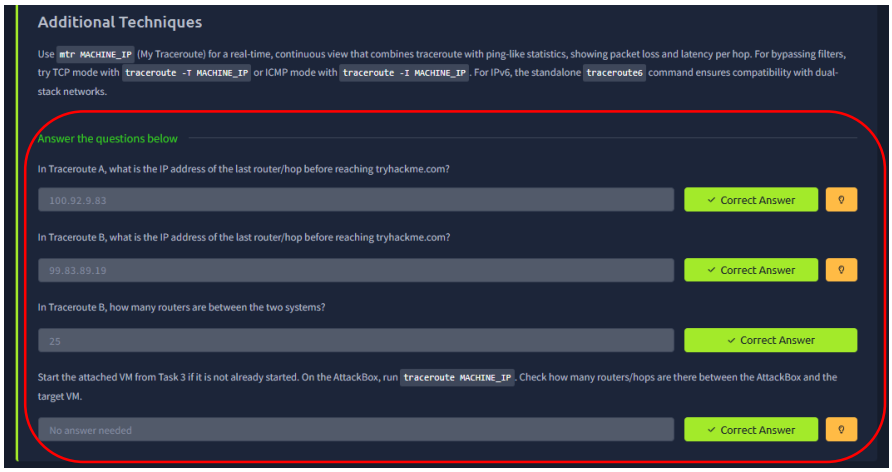

- **Fig. 2.4:** Utilizing Shodan.io to passively enumerate global Apache and Nginx server statistics.



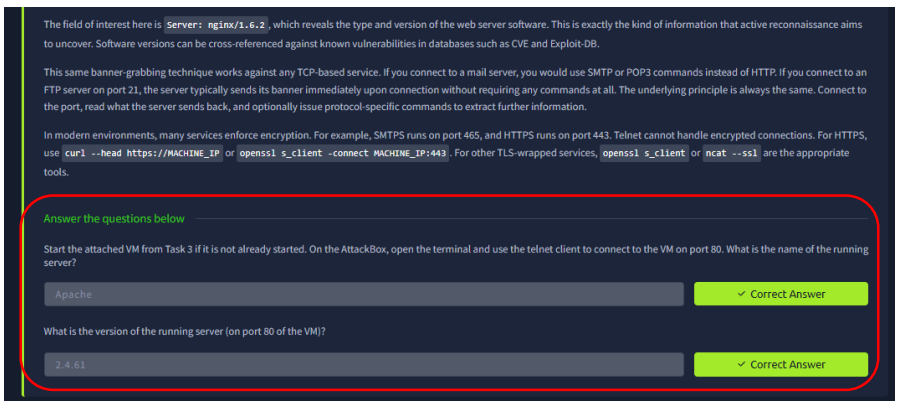

- **Fig. 2.5:** TryHackMe Active Reconnaissance room completion certificate.



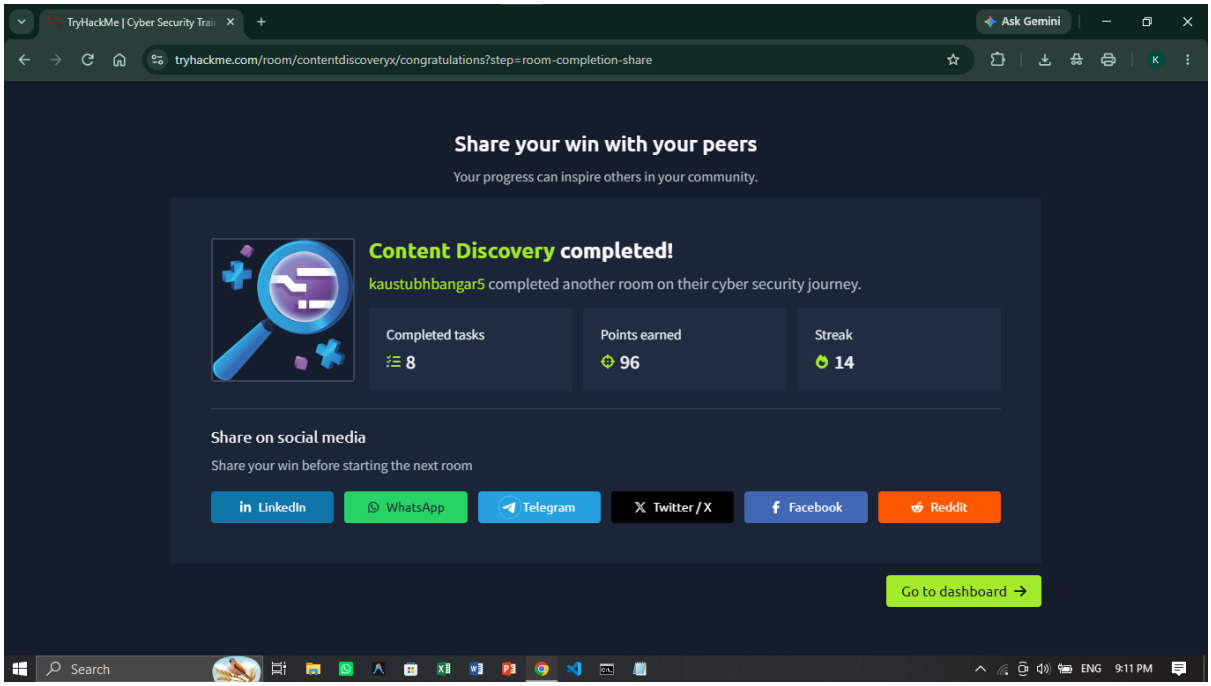

- **Fig. 2.6:** Active Reconnaissance task dashboard confirming full module completion.



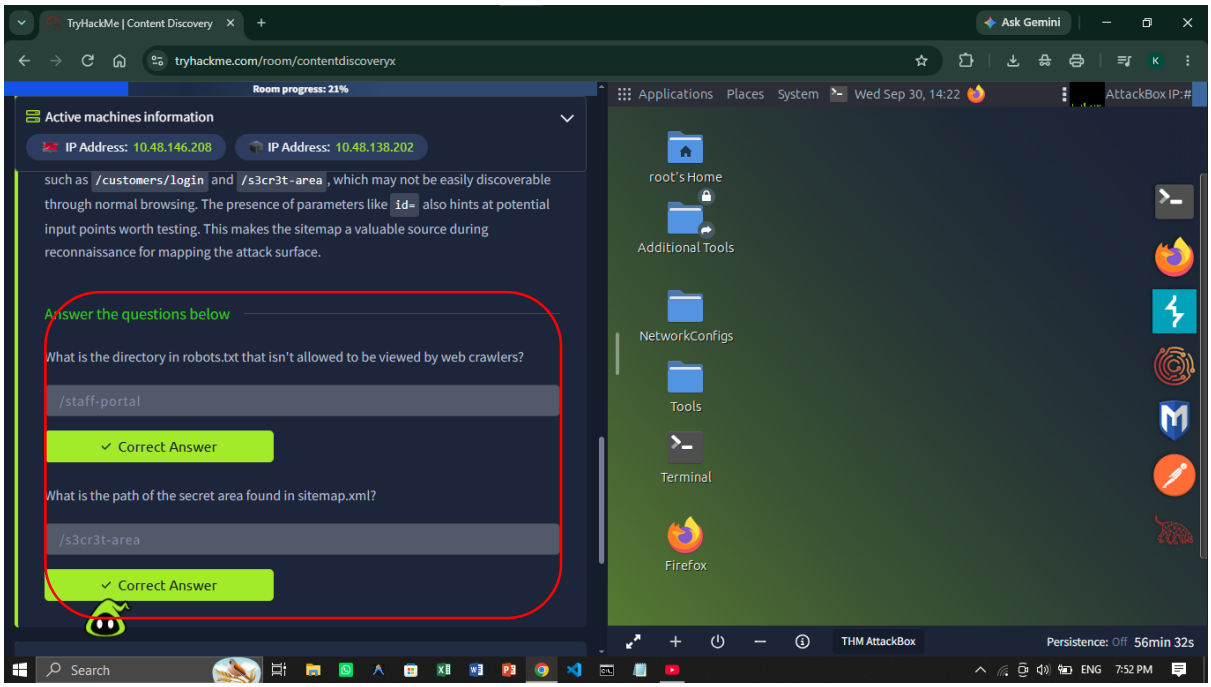

- **Fig. 2.7:** Analysis of `traceroute` network hops identifying final router IPs and hop counts.



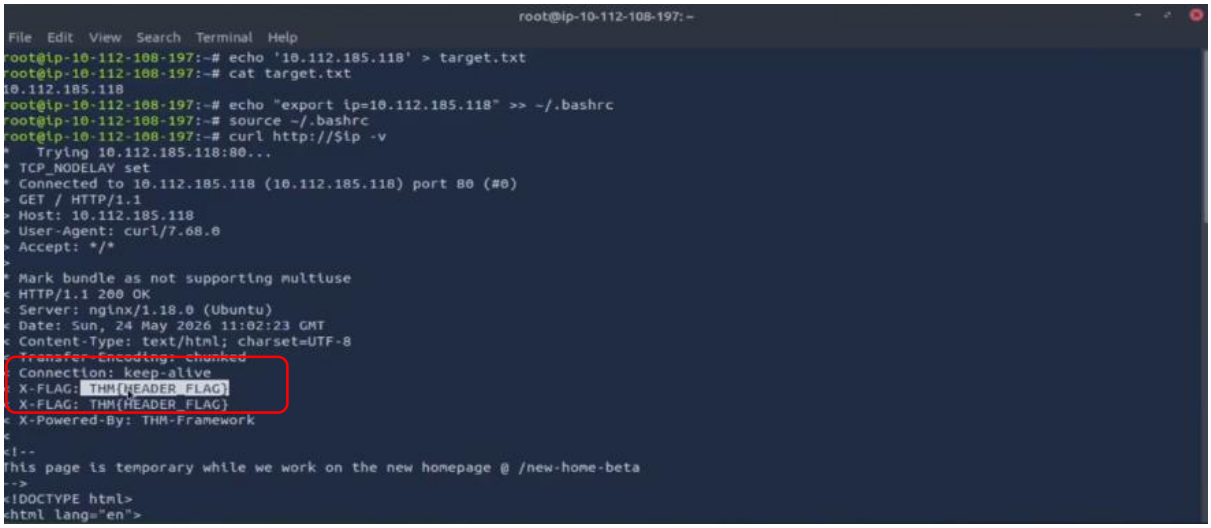

- **Fig. 2.8:** Active banner grabbing execution via `telnet` revealing the `Apache/2.4.61` software version.



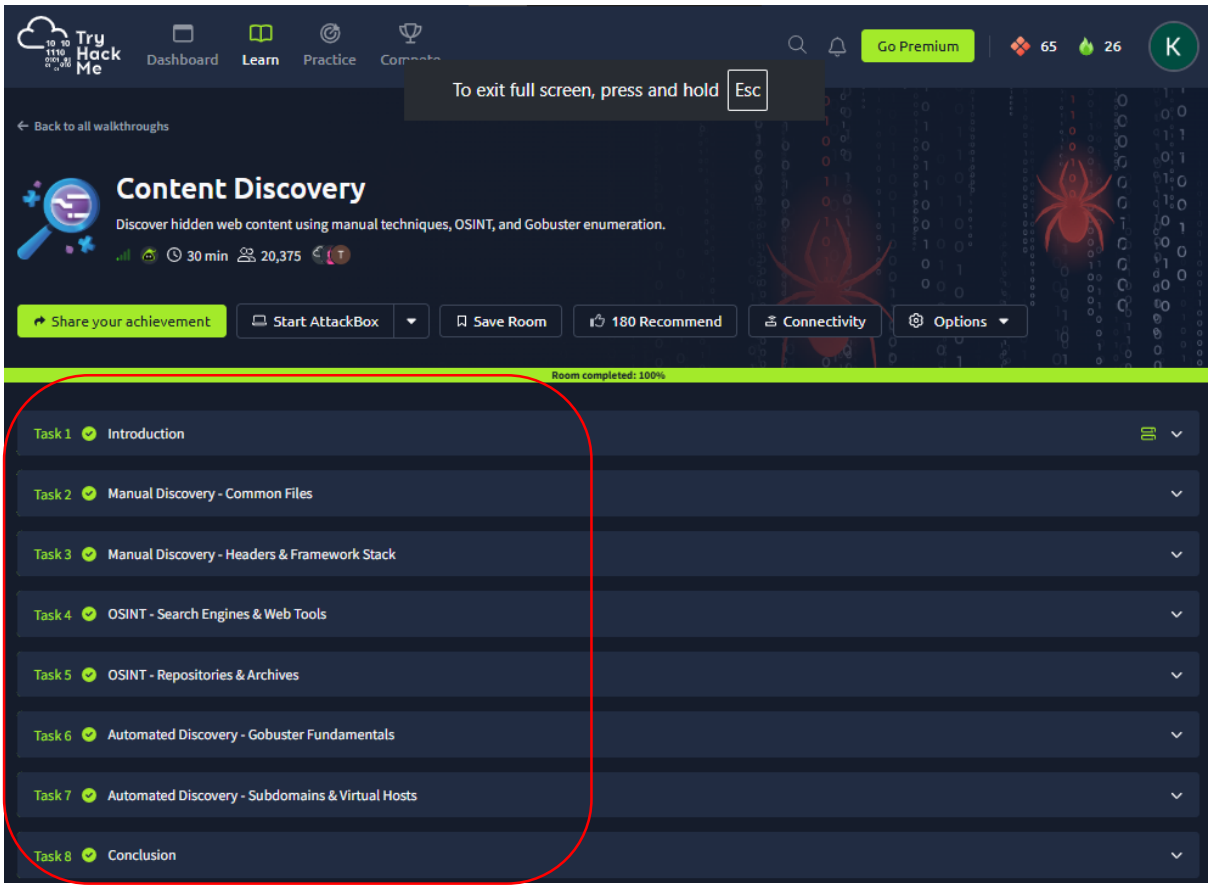

- **Fig. 2.9:** TryHackMe Content Discovery room completion certificate.




- **Fig. 2.10:** Manual discovery of `/staff-portal` via `robots.txt` and `/s3cr3t-area` via `sitemap.xml`.




- **Fig. 2.11:** Manual header inspection via `curl` revealing the `THM{HEADER_FLAG}` custom header.




- **Fig. 2.12:** Content Discovery task dashboard confirming the completion of manual, OSINT, and Gobuster automated enumeration modules.

## 7. Results

Task 2 was completed successfully. By systematically combining passive OSINT, active network probing, and targeted content discovery methodologies, I successfully constructed a comprehensive profile of the simulated target environments. I effectively utilized public databases and DNS logs to enumerate infrastructure without triggering detection alerts, transitioned to active methods to trace network pathways and extract exact software version banners directly from live services, and successfully uncovered hidden administrative directories and custom HTTP headers through manual inspection.

## 8. Key Learnings

Completing this multi-faceted reconnaissance module provided several critical insights into the methodology of professional penetration testing:

- **The Power of Passive Intelligence:** A massive amount of actionable intelligence can be gathered without ever touching the target's servers. Certificate Transparency logs and Shodan are highly effective at revealing forgotten subdomains and exposed staging environments that automated active scanners might miss.
- **DNS is a Treasure Trove:** Beyond translating IP addresses, DNS `TXT` and `MX` records expose third-party integrations, email infrastructure, and internal domain verification strings.
- **Active Probing Requires Authorization:** Unlike Shodan queries, utilizing `ping`, `traceroute`, or `telnet` directly against a target generates immediate server and firewall logs. This activity must strictly adhere to the defined Rules of Engagement (RoE) during a professional assessment.
- **Banners Lead Directly to Exploits:** The simple act of grabbing a banner using `telnet` or `nc` to identify a specific software version is often the most direct path to finding a critical CVE. If a server broadcasts an outdated, vulnerable banner (like an unpatched Apache daemon), an analyst can immediately transition from reconnaissance to targeted exploitation.

- **Security Through Obscurity Fails:** Relying on unlinked directories or omitting pages from HTML navigation does not protect sensitive content. Automated tools like Gobuster paired with robust wordlists, or simple manual checks of `robots.txt` and `sitemap.xml`, trivially defeat obscurity-based defenses.

## 9. Conclusion

Reconnaissance and Enumeration fundamentally dictate the success and trajectory of a penetration test. The completion of the TryHackMe modules for Passive Reconnaissance, Active Reconnaissance, and Content Discovery solidified the necessity of a structured, phased methodology. By starting broadly with stealthy, passive OSINT techniques (WHOIS, DNS, Shodan) to build an initial profile, gradually narrowing the focus with active network probing (Traceroute, Telnet) to confirm live assets, and finally applying rigorous content discovery techniques (Manual inspection, Gobuster) to map the application layer, an analyst can comprehensively map the complete attack surface of an organization. This multi-layered intelligence-gathering phase provides the critical context and target lists required to effectively configure interception tools like Burp Suite and execute highly targeted vulnerability testing in the subsequent phases of the WAPT internship.

## 10. References


1. TryHackMe - Passive Reconnaissance: https://tryhackme.com/room/passiverecon
2. TryHackMe - Active Reconnaissance: https://tryhackme.com/room/activerecon
3. TryHackMe - Content Discovery: https://tryhackme.com/room/contentdiscoveryx
4. WAPT Phase 1 Task Instructions & Guidelines - IqraSec Academy